

Portfolio Web Application — Project Documentation

Author: Arijeet Ghosal

Version: 1.0.0

Date: July 2026

License: ISC

Table of Contents

1. [Executive Summary](#1-executive-summary)
2. [Project Overview](#2-project-overview)
3. [Technology Stack](#3-technology-stack)
4. [Directory Structure & Architecture](#4-directory-structure--architecture)
5. [Build Pipeline](#5-build-pipeline)
6. [Frontend Architecture](#6-frontend-architecture)
 - 6.1 [HTML Pages](#61-html-pages)
 - 6.2 [CSS Design System](#62-css-design-system)
 - 6.3 [JavaScript Modules](#63-javascript-modules)
7. [AI Chatbot System](#7-ai-chatbot-system)
 - 7.1 [Client-Side Architecture](#71-client-side-architecture)
 - 7.2 [Server-Side API](#72-server-side-api)
 - 7.3 [Animated Avatar Engine](#73-animated-avatar-engine)
 - 7.4 [Knowledge Base & Fallback System](#74-knowledge-base--fallback-system)
8. [Visual Effects & Animations](#8-visual-effects--animations)
9. [Media & Video Integration](#9-media--video-integration)
10. [Deployment & Hosting](#10-deployment--hosting)
11. [SEO & Performance](#11-seo--performance)
12. [Security Considerations](#12-security-considerations)
13. [Rules & Coding Conventions](#13-rules--coding-conventions)

14. [File Reference Index](#14-file-reference-index)

15. [Appendices](#15-appendices)

1. Executive Summary

This document provides a comprehensive, thesis-level technical specification of the **Arijeet Ghosal Portfolio Web Application**. The project is a full-stack, production-grade personal portfolio website featuring a custom-built AI chatbot, animated 3D avatar, interactive visual effects, project demonstration videos, and a zero-dependency build optimization pipeline.

The application is designed to showcase professional expertise in data engineering, artificial intelligence, and cloud technologies while serving as a functional platform for client engagement, service booking, and real-time AI-powered conversation.

Key Highlights:

- Zero-framework, vanilla HTML/CSS/JS frontend for maximum performance
 - Dual-provider AI chatbot (OpenAI GPT-3.5 + Google Gemini 1.5 Flash) with offline fallback
 - Custom SVG robot avatar with breathing, talking, yawning, and sleeping animations
 - WebAudio API synthesized sound effects (yawn/snore)
 - Python-based build pipeline achieving ~30% code reduction
 - Vercel serverless deployment with environment variable management
 - Mouse sparkle trail, neural particle canvas, and magnetic cursor effects
-

2. Project Overview

2.1 Purpose

The portfolio serves three primary objectives:

1. **Professional Showcase:** Present Arijeet Ghosal's experience at Bosch and Microsoft, certifications (Azure, Databricks, TensorFlow, AWS, GCP), and technical projects.
2. **Client Engagement:** Provide a services marketplace with tiered pricing (\$19–\$99) and direct contact channels.
3. **Technical Demonstration:** The website itself demonstrates advanced frontend engineering, AI integration, and production deployment skills.

2.2 Target Audience

- Potential employers and recruiters
- Freelance clients seeking data engineering and AI services
- Academic peers and collaborators
- Technical community members

2.3 Core Features

FEATURE	DESCRIPTION
Hero Landing	Animated introduction with typing effect, stats counters, and CTA buttons
About Section	Professional biography with info cards
Experience Timeline	Interactive career history (Bosch, Microsoft)
Projects Gallery	Project cards with embedded video demonstrations
Skills Radar	Interactive skill visualization with animated radar chart
Certifications Grid	9 professional certification cards with detail modals
Services Marketplace	Tiered service pricing with booking CTAs
AI Chatbot	Full conversational AI with voice I/O and animated avatar
Stress Relief Games	Interactive mini-games page for engagement
Mouse Sparkle Trail	Glitter particles following cursor movement

3. Technology Stack

3.1 Frontend

TECHNOLOGY	PURPOSE	VERSION/DETAILS
HTML5	Page structure and semantics	Semantic elements, ARIA labels
CSS3	Styling, animations, responsive design	Custom properties, keyframes, grid, flexbox
Vanilla JavaScript	Interactivity, effects, chatbot client	ES6+ classes, async/await, Web APIs
Google Fonts	Typography	Space Grotesk, Inter

3.2 Backend / API

TECHNOLOGY	PURPOSE
Vercel Serverless Functions	API endpoint hosting
Node.js (ESM)	Serverless runtime
OpenAI API (GPT-3.5 Turbo)	Primary AI provider
Google Gemini API (1.5 Flash)	Fallback AI provider with Google Search Grounding

3.3 Build Tools

TOOL	PURPOSE
Python 3	Build pipeline script (`scripts/build.py`)
Regular Expressions	HTML/CSS/JS minification
shutil	Binary asset copying

3.4 Web APIs Used

API	PURPOSE
Web Speech API (`SpeechRecognition`)	Voice-to-text input for chatbot
Web Speech API (`SpeechSynthesis`)	Text-to-speech output for chatbot responses
Web Audio API (`AudioContext`)	Synthesized yawn and snore sound effects
Canvas API	Neural network particle background, click particles
IntersectionObserver API	Scroll-triggered section reveal animations
localStorage API	Chatbot memory, feedback logging, visit tracking
Fetch API	API communication with OpenAI/Gemini endpoints

3.5 Deployment

SERVICE	PURPOSE
Vercel	Production hosting and serverless function deployment
GitHub	Version control and CI/CD trigger

4. Directory Structure & Architecture

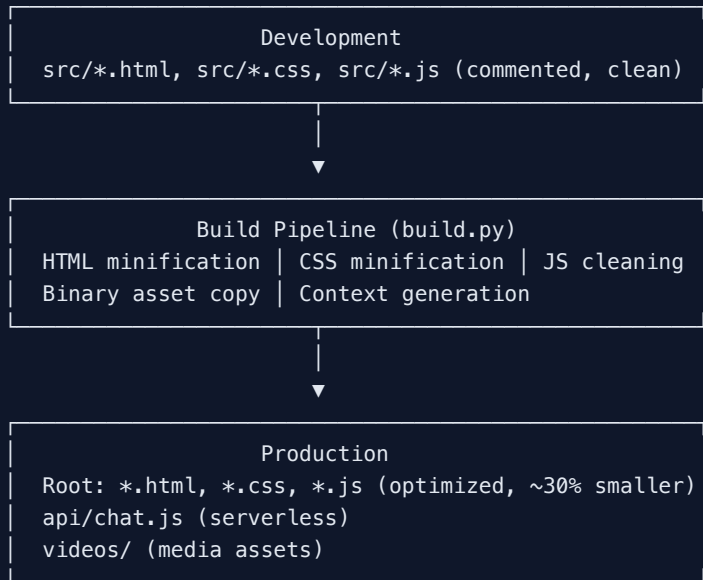
```

Portfolio-main/
├── src/                                # Source development files (readable, commented)
│   ├── index.html                     # Main portfolio page (~1094 lines)
│   ├── projects.html                  # Project hub sub-page
│   ├── services.html                  # Services marketplace sub-page
│   ├── games.html                     # Stress relief games sub-page
│   ├── style.css                      # Primary stylesheet (~3560 lines)
│   ├── swiss-override.css             # Design system overrides
│   ├── avatar3d.css                   # Chatbot avatar animations
│   ├── script.js                      # Main JavaScript (~1180 lines)
│   ├── common.js                      # Shared effects (neural canvas, cursor)
│   ├── avatar3d.js                    # Chatbot avatar SVG engine
│   ├── projects.js                    # Projects page interactivity
│   ├── services.js                    # Services page interactivity
│   ├── Avatar.jpg                     # Legacy avatar image
│   ├── chatbot_icon.png               # Chatbot favicon and toggle icon
│   ├── metahuman_avatar.png           # Metahuman avatar image
│   └── riya-anime-avatar.png          # Legacy anime avatar
├── api/                                # Vercel serverless functions
│   └── chat.js                         # AI chatbot API handler (~158 lines)
├── scripts/                            # Build and utility scripts
│   └── build.py                       # Python build pipeline (~182 lines)
├── videos/                             # Project demonstration media
│   ├── uber_bi.mp4                    # Uber Dashboard demo
│   ├── credit_card_video.mp4          # Credit Card Dashboard demo
│   ├── voice_bot.mp4                  # Voice Bot demo
│   ├── complaint_bot.mp4              # Complaint Bot demo
│   ├── nvh_predictability.mp4         # NVH Analysis demo
│   ├── dbscan.mp4                     # DBSCAN Clustering demo
│   ├── *_thumbnail.png                # AI-generated video poster images
│   └── *.webp                         # Browser-captured walkthrough recordings
├── .context/                           # Build output
│   └── codebase_context.md            # Token-optimized codebase snapshot
├── *.html, *.css, *.js                 # Production-built (minified) web assets
├── vercel.json                         # Vercel deployment configuration
├── .gitignore                          # Git file exclusion rules
├── package.json                        # Project metadata
└── chatbot_icon.png                    # Production favicon

```

4.1 Architecture Pattern

The project follows a **Source** → **Build** → **Production** pipeline pattern:



4.2 Data Flow

```
User Browser → index.html (static assets)
|
├─ Neural Canvas (common.js) → <canvas> rendering
├─ UI Effects (script.js) → DOM manipulation
├─ Chatbot Input → POST /api/chat → OpenAI / Gemini
|                                     ↳ JSON response
├─ Chatbot Offline → Local Knowledge Base (script.js)
├─ Voice Input → SpeechRecognition API
├─ Voice Output → SpeechSynthesis API
└─ Avatar Animation → avatar3d.js + avatar3d.css
```

5. Build Pipeline

5.1 Overview

The build pipeline is implemented in `scripts/build.py` — a zero-dependency Python 3 script that transforms readable source code into optimized production assets.

Command:

```
python3 scripts/build.py
```

Or via npm:

```
npm run build
```

5.2 Pipeline Stages

Stage 1: Binary Asset Copy

```
BINARY_FILES = ['metahuman_avatar.png', 'Avatar.jpg', 'riya-anime-avatar.png', 'chatbot_icon.png']
```

All image assets from `src/` are copied directly to the project root using `shutil.copy2()` (preserving metadata).

Stage 2: HTML Minification

```
def minify_html(html):  
    minified = re.sub(r'<!--[\s\S]*?-->', '', html)      # Remove HTML comments  
    minified = re.sub(r'\s+', ' ', minified)              # Collapse whitespace  
    minified = re.sub(r'>\s+<', '><', minified)          # Remove inter-tag spaces  
    return minified.strip()
```

- Removes all HTML comments (`<!-- -->`)
- Collapses all whitespace sequences into single spaces
- Removes whitespace between HTML tags
- **Typical reduction:** ~36%

Stage 3: CSS Minification

```
def minify_css(css):  
    minified = re.sub(r'/\s*\s*\s*/', '', css)           # Remove CSS comments  
    minified = re.sub(r'\s+', ' ', minified)              # Collapse whitespace  
    minified = re.sub(r'\s*([:;])\s*', r'\1', minified)  # Remove operator spaces  
    return minified.strip()
```

- Removes all block comments (`/* ... */`)
- Collapses whitespace
- Strips spaces around CSS delimiters (`{`, `}`, `:`, `;`, `,`)
- **Typical reduction:** ~29%

Stage 4: JavaScript Cleaning

```
def clean_js(js):
    minified = re.sub(r'\/\*[\\s\\S]*?\\*/', '', js)          # Remove block comments
    # Line-by-line processing:
    # - Skip empty lines
    # - Skip full-line comments (//)
    # - Safely remove trailing comments (checking for string context)
    return '\\n'.join(cleaned_lines)
```

- Removes block comments
- Strips full-line single-line comments
- Safely removes trailing comments while respecting string literals (checks quote parity)
- Preserves URLs containing `://`
- **Typical reduction:** ~29%

Stage 5: Context Generation

Generates `.context/codebase_context.md` — a unified, token-optimized snapshot of the entire codebase for AI assistant consumption. This file concatenates all optimized source files into a single markdown document with code fences.

5.3 Build Output Statistics

FILE	SOURCE SIZE	BUILT SIZE	REDUCTION
index.html	67.1 KB	42.9 KB	36.1%
style.css	77.3 KB	55.1 KB	28.8%
script.js	51.0 KB	36.4 KB	28.6%
swiss-override.css	21.8 KB	14.5 KB	33.6%
avatar3d.js	9.5 KB	8.0 KB	15.9%
Total	**~323 KB**	**~226 KB**	**~30%**

5.4 Build Rules

1. **Source files are NEVER served directly** — only built output is served in production.
2. **All edits must be made in** `src/` — the root files are overwritten on each build.
3. **Binary files must be registered** in the `BINARY_FILES` list to be copied.
4. The `api/` directory is **NOT processed** by the build pipeline (Vercel handles it directly).

5. The **videos/** directory is NOT processed — media assets are served as-is.

6. Frontend Architecture

6.1 HTML Pages

6.1.1 `index.html` — Main Portfolio (~1094 lines)

The primary landing page containing all portfolio sections:

SECTION	LINES	DESCRIPTION
Preloader	19–36	SVG ring animation with progress bar
Scroll Progress	38–39	Fixed top bar showing page scroll position
Neural Canvas	41–42	Background particle network animation
Click Canvas	44–45	Explosion particles on mouse click
Scanline Overlay	47–48	CRT-style horizontal line sweep effect
Custom Cursor	50–52	Magnetic dot cursor with trailing ring
Side Navigation	54–63	Dot-based section jump navigation
Back-to-Top	65–68	Floating scroll-to-top button
Navigation Bar	70–92	Fixed header with page links
Hero Section	93–158	Name, title, typing effect, stats, CTAs
About Section	159–241	Bio text with info icon cards
Experience Section	242–320	Career timeline (Bosch, Microsoft)
Projects Section	321–471	Project cards with video players
Skills Section	472–629	Skill bars + interactive radar chart
Certifications	630–929	9 certification cards with details
Services Teaser	930–964	CTA for services page
Contact Section	965–1013	Email, LinkedIn, GitHub, location cards
Footer	1014–1031	Copyright and social links
AI Chatbot Widget	1042–1086	Chatbot container, toggle, window

6.1.2 `projects.html` — Project Hub

Categorized project directory with filtering. Uses inline CSS overrides to ensure visibility (since `script.js` is not loaded on sub-pages).

6.1.3 `services.html` — Services Marketplace

Detailed service listings with:

- Data Analysis (\$19)

- Data Visualization (\$29)
- Data Engineering (\$49)
- AI/Dev Builds (\$99)

6.1.4 `games.html` — Stress Relief Games

Interactive mini-games including breathing exercises and click-based games. Self-contained with inline JavaScript and styles.

6.2 CSS Design System

6.2.1 `style.css` — Primary Stylesheet (~3560 lines)

Design Tokens (CSS Custom Properties):

```
:root {
  --primary: #00f0ff;          /* Cyan accent */
  --primary-glow: rgba(0, 240, 255, 0.3);
  --accent: #8b5cf6;          /* Purple accent */
  --bg-primary: #07131d;      /* Dark navy background */
  --bg-card: #0d1b2a;         /* Card backgrounds */
  --text-primary: #e2e8f0;     /* Light text */
  --text-secondary: #94a3b8;   /* Muted text */
  --ease-spring: cubic-bezier(0.175, 0.885, 0.32, 1.275);
}
```

Key CSS Architecture:

- **Scroll-triggered animations:** All `.section` elements start at `opacity: 0; transform: translateY(40px)` and transition to visible via the `.visible` class added by IntersectionObserver.
- **Card spotlight effect:** Cards have a radial gradient `::before` pseudo-element that follows mouse position.
- **Responsive breakpoints:** `480px`, `768px`, `1024px`, `1200px`

6.2.2 `swiss-override.css` — Design Overrides (~550 lines)

Provides Swiss/Bauhaus-inspired design enhancements:

- Enhanced typography scales
- Refined spacing and grid systems
- Label and badge styling
- Enhanced button treatments

6.2.3 `avatar3d.css` — Avatar Animations (~65 lines)

Defines all chatbot avatar animation keyframes:

ANIMATION	TRIGGER	EFFECT
`riyaFloat`	Always (idle)	Gentle vertical bobbing
`riyaTalk`	`.riya-speaking`	Mouth scale oscillation
`riyaExpress`	`.riya-speaking`	Eye micro-movement
`riyaNod`	`.riya-speaking`	Head nodding
`riyaYawn`	`.riya-yawn`	Mouth wide open stretch
`riyaSleepy`	`.riya-yawn`	Eyes close
`riyaDeepBreathe`	`.riya-sleeping-mode`	Slow body rise/fall
`zzzFloat`	`.riya-sleeping-mode`	Floating "Zzz..." text
`sparkleLife`	Mouse movement	Glitter particle fade-out

6.3 JavaScript Modules

6.3.1 `script.js` — Main Application (~1180 lines)

Contains the following classes and systems:

CLASS/SYSTEM	LINES	PURPOSE
`Preloader`	~1–50	Manages loading animation and fade-out
`HeroTyping`	~50–90	Typewriter effect for role titles
`StatsCounter`	~90–140	Animated number counting for hero stats
`ScrollObserver`	~140–200	IntersectionObserver for section reveals
`RadarChart`	~200–350	Canvas-based skill visualization
`TiltEffect`	~350–400	3D card tilt on mouse hover
`MagneticButtons`	~400–450	Buttons that attract toward cursor
`Chatbot`	~450–1080	Full AI chatbot client (see Section 7)
Sparkle Trail	~1140–1180	Mouse glitter particle spawner

6.3.2 `common.js` — Shared Effects (~313 lines)

CLASS	PURPOSE
`NeuralCanvas`	Animated particle network background with mouse interaction
`ClickParticles`	Explosion particle effect on mouse click
`CustomCursor`	Magnetic custom cursor with dot + ring
`ScrollProgress`	Top progress bar tracking page scroll
`BackToTop`	Floating back-to-top button visibility
`Parallax`	Parallax scrolling effect on hero elements
`CardSpotlight`	Mouse-tracking radial gradient on cards
`KeyboardNav`	Keyboard accessibility (Tab focus ring)

6.3.3 `avatar3d.js` — Avatar Engine (~220 lines)

See Section 7.3 for detailed documentation.

6.3.4 `projects.js` & `services.js`

Page-specific scripts that initialize shared UI components (section reveals, navigation) on sub-pages that don't load `script.js`.

7. AI Chatbot System

The chatbot is the most complex subsystem, spanning three files and integrating multiple Web APIs.

7.1 Client-Side Architecture

Class: `Chatbot` (in `script.js`)

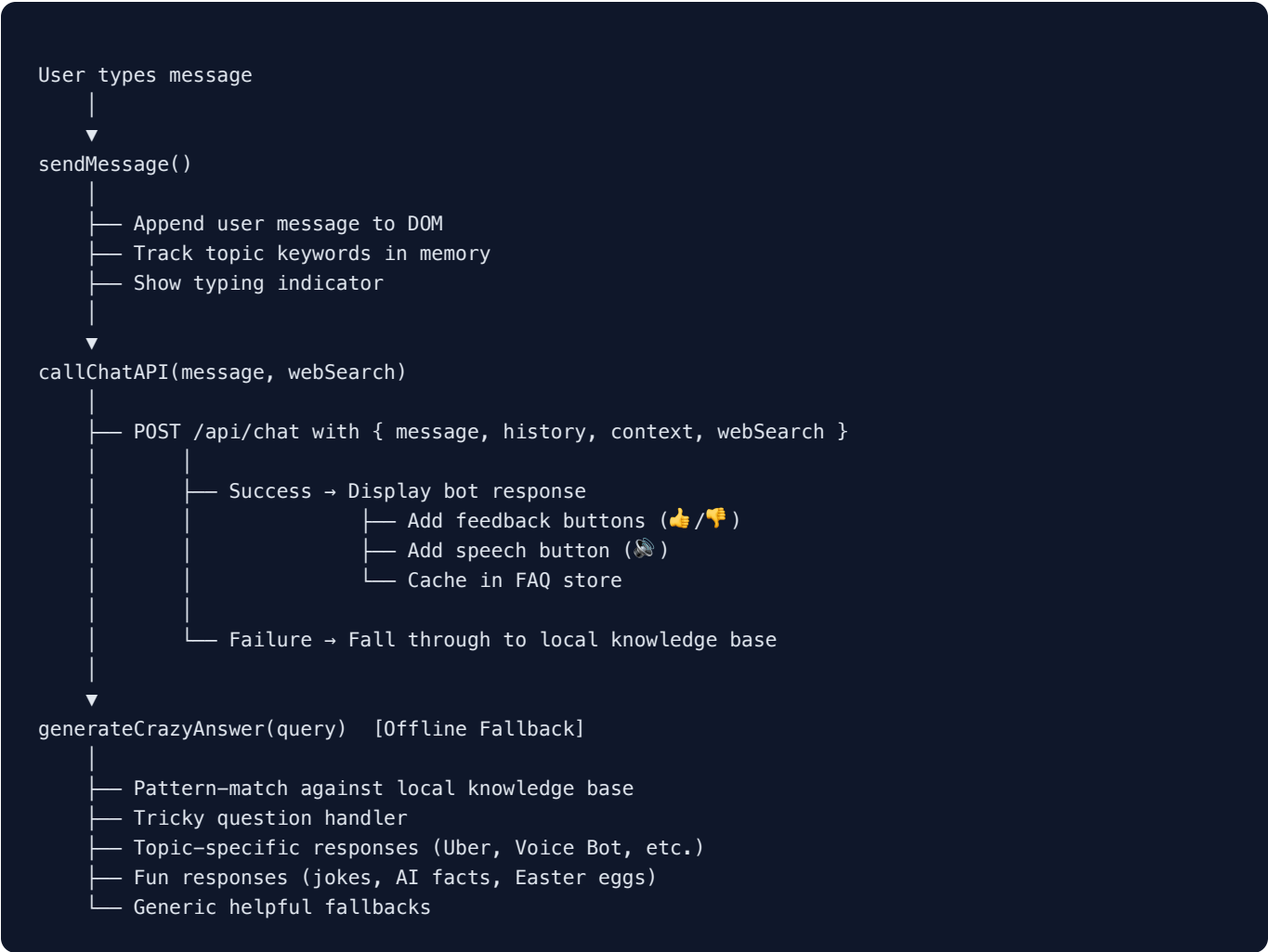
7.1.1 Initialization

```
class Chatbot {
  constructor() {
    this.STORAGE_KEY = 'riya_memory';
    this.FAQ_KEY = 'riya_faq';
    this.FEEDBACK_KEY = 'riya_feedback';
    // ... DOM element references, localStorage loading
  }
}
```

localStorage Keys:

KEY	PURPOSE
`riya_memory`	Visit count, topic frequency map, last visit timestamp
`riya_faq`	Cached question/answer pairs for offline retrieval
`riya_feedback`	User thumbs-up/down ratings on responses

7.1.2 Message Flow



7.1.3 Voice Input (Speech-to-Text)

```
// Uses Web Speech API
this.recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
this.recognition.lang = 'en-US';
this.recognition.interimResults = false;
```

- Microphone button (🎤) toggles recording
- Recording state shows red pulsing animation
- Transcribed text is auto-sent as a message

7.1.4 Voice Output (Text-to-Speech)

```
speakResponse(text) {
  const utterance = new SpeechSynthesisUtterance(cleanText);
  utterance.rate = 1.05;
  utterance.pitch = 1.0;
  window.speechSynthesis.speak(utterance);
}
```

- Speech button (🗣️) on each bot response
- Triggers avatar speaking animation via `riya:speech-start` event
- Cleans HTML tags and emojis before speaking

7.2 Server-Side API

File: `api/chat.js`

Runtime: Vercel Serverless Function (Node.js ESM)

Endpoint: `POST /api/chat`

7.2.1 Request Schema

```
{
  "message": "string",
  "history": [
    { "role": "user|assistant", "content": "string" }
  ],
  "context": "string (optional session context)",
  "webSearch": true | false
}
```

7.2.2 Provider Cascade

1. OpenAI GPT-3.5 Turbo (Primary)

└─ Model: gpt-3.5-turbo

└─ Temperature: 0.85

└─ Max Tokens: 180

└─ Conversation history: last 4 messages
2. Google Gemini 1.5 Flash (Fallback)

└─ Model: gemini-1.5-flash

└─ Temperature: 0.85

└─ Max Tokens: 220

└─ Google Search Grounding (when webSearch=true)

└─ System instruction support
3. HTTP 500 Error (Both fail)

└─ Client falls back to local knowledge base

7.2.3 System Prompt Architecture

The system prompt contains:

- **Identity:** "You are Chatbot, a polished, professional portfolio assistant"
- **Rules:** Never refuse, always answer confidently, keep fresh wording
- **Detailed Facts:** Full biography, skills, projects, certifications, contact, pricing
- **Tricky Question Handling:** Example Q&A pairs for indirect questions
- **Behavioral Constraints:** No session stats, no repetition

7.2.4 Environment Variables

VARIABLE	PROVIDER	REQUIRED
`OPENAI_API_KEY`	OpenAI	Optional (primary)
`GEMINI_API_KEY`	Google	Optional (fallback)

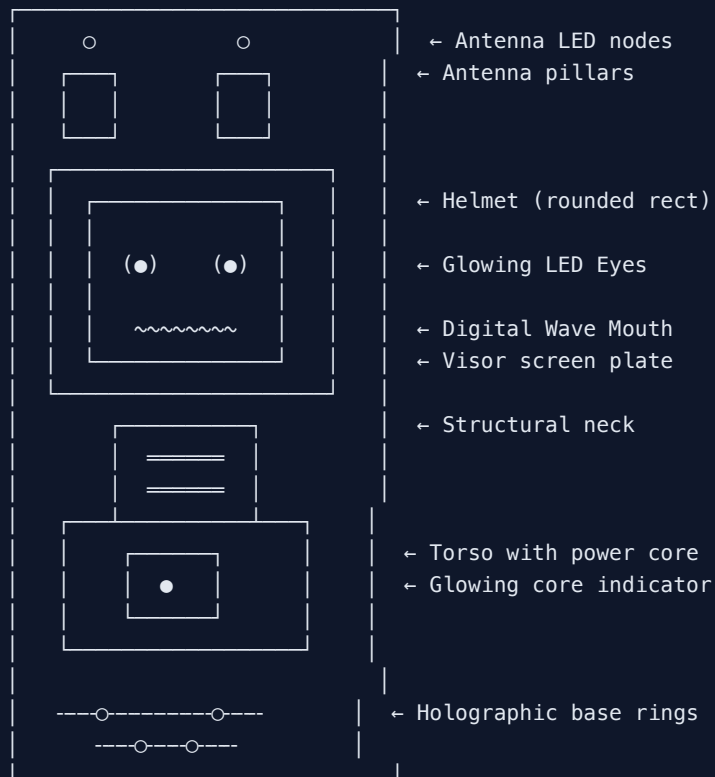
At least one must be configured for online AI responses.

7.3 Animated Avatar Engine

File: `avatar3d.js`

7.3.1 SVG Robot Design

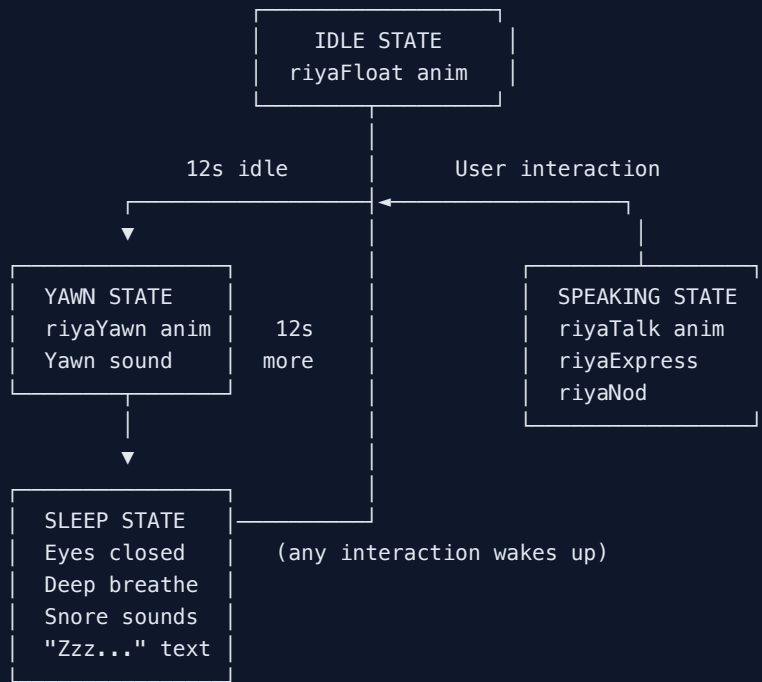
The avatar is a vector SVG robot assistant rendered inline:



SVG Features:

- Linear gradients (`bodyG` : dark navy, `glowG` : cyan-to-purple)
- Gaussian blur glow filter (`glowF`)
- Concentric holographic base ellipses with dash patterns

7.3.2 State Machine



Sound Guard: All sounds are muted when the chatbot window is closed

```
( !chat.classList.contains('open') ).
```

7.3.3 Audio Synthesis

Yawn Sound:

```
// Triangle oscillator, descending frequency 280Hz → 75Hz over 2 seconds
osc.type = 'triangle';
osc.frequency.setValueAtTime(280, audioCtx.currentTime);
osc.frequency.exponentialRampToValueAtTime(75, audioCtx.currentTime + 2.0);
```

Snore Sound:

```
// Sawtooth oscillator at 45Hz through bandpass filter at 140Hz
// Simulates rhythmic inhale/exhale breathing pattern
osc.type = 'sawtooth';
osc.frequency.setValueAtTime(45, audioCtx.currentTime);
// Gain envelope: 0 → 0.08 (inhale) → 0.001 (exhale)
```

7.4 Knowledge Base & Fallback System

When the API is unavailable, the chatbot uses a local pattern-matching knowledge base in

```
generateCrazyAnswer(query) :
```

PATTERN KEYWORDS	RESPONSE CATEGORY
`plan`, `master`, `world`	Humorous master plan responses
`know him`, `how well`, `any good`, `can he`	Tricky question handler (confident positive answers)
`who` + `arijeet`/`ghosal`/`creator`	About Arijeet responses
`uber`, `taxi`, `ride`	Uber Dashboard project details
`fact`, `crazy`, `science`	AI fun facts
`joke`, `funny`, `laugh`	AI-themed jokes
`hire`, `service`, `price`	Services and pricing info
`skill`, `tech`, `stack`	Technical skills overview
`news`, `latest`, `update`	AI news (with web search grounding)
`build`, `pipeline`, `architect`	Architecture guidance
(no match)	Helpful fallback with topic suggestions

8. Visual Effects & Animations

8.1 Neural Network Particle Canvas

File: `common.js`, Class: `NeuralCanvas`

- Renders 50–100 floating particles on `<canvas>` element
- Particles connect with lines when within 140px distance
- Mouse interaction: particles within 180px radius are repelled
- Color scheme: cyan (`#00f0ff`) and purple (`#8b5cf6`)
- Runs at 60fps via `requestAnimationFrame`

8.2 Click Particle Explosions

File: `common.js`, Class: `ClickParticles`

- On mouse click, spawns 12 particles radiating outward
- Particles have random velocities, sizes, and colors
- Gravity simulation pulls particles downward
- 2-second lifetime with opacity fade

8.3 Custom Magnetic Cursor

File: `common.js`, Class: `CustomCursor`

- Replaces default cursor with a small dot + larger trailing ring
- Ring smoothly follows dot with easing (lerp factor 0.15)
- Both scale up on hover over interactive elements
- Hidden on mobile devices (pointer: coarse)

8.4 Mouse Sparkle Trail

File: `script.js` (bottom)

- Spawns 1–2 glowing particles per mouse movement event
- Throttled to ~25fps for performance
- 6 color palette: cyan, purple, green, pink, gold, white
- Random size (3–9px), random drift direction
- CSS custom properties `--sx` and `--sy` control drift vector
- Particles self-remove after 0.7s animation

8.5 Scroll-Triggered Section Reveals

File: `script.js`, Function: `initScrollObserver`

- Uses `IntersectionObserver` with `threshold: 0.15`
- Adds `.visible` class when section enters viewport
- CSS transition: `opacity 0→1`, `translateY 40px→0`
- Children (cards, timeline items) stagger with `transition-delay`

8.6 Card Spotlight Effect

File: `common.js`, Function: `initCardSpotlight`

- Tracks mouse position over `.project-card` elements
- Sets `--mouse-x` and `--mouse-y` CSS custom properties

- `::before` pseudo-element renders radial gradient at cursor position
- Creates a "flashlight" hover effect

9. Media & Video Integration

9.1 Video Player Configuration

All project demonstration videos use native HTML5 `<video>` elements:

```
<video controls preload="metadata" poster="videos/uber_thumbnail.png"
        style="width: 100%; border-radius: 12px;">
  <source src="videos/uber_bi.mp4" type="video/mp4">
</video>
```

9.2 Video Assets

VIDEO	PROJECT	FILE SIZE	THUMBNAIL
`uber_bi.mp4`	Uber Data Dashboard	6.9 MB	AI-generated dark dashboard
`credit_card_video.mp4`	Credit Card Analytics	4.3 MB	AI-generated platinum card UI
`voice_bot.mp4`	Voice Bot Assistant	101 MB	AI-generated audio waveform
`complaint_bot.mp4`	NLP Complaint Bot	21.4 MB	AI-generated robot chat UI
`nvh_predictability.mp4`	NVH Signal Analysis	7.8 MB	AI-generated vibration graph
`dbscan.mp4`	DBSCAN Clustering	27.6 MB	AI-generated cluster viz

9.3 Thumbnail Generation

All video thumbnails were generated using AI image generation with custom prompts describing dark-themed, futuristic dashboard mockups matching the portfolio's design language.

10. Deployment & Hosting

10.1 Vercel Configuration

The project is deployed on **Vercel** with the following configuration:

- **Framework Preset:** Other (static site)
- **Build Command:** `python3 scripts/build.py`
- **Output Directory:** `.` (root directory — configured via `vercel.json`)
- **Serverless Functions:** `api/chat.js` auto-detected by Vercel

10.1.1 Vercel Configuration File (`vercel.json`)

To prevent the "No Output Directory named 'public' found" error during automated deployment, a `vercel.json` file is placed in the root directory:

```
{
  "outputDirectory": "."
}
```

This configuration instructs Vercel that the static output generated by the build pipeline resides in the root directory, ensuring successful deployments.

10.2 Environment Variables (Vercel Dashboard)

VARIABLE	DESCRIPTION
<code>`OPENAI_API_KEY`</code>	OpenAI API key for GPT-3.5 Turbo
<code>`GEMINI_API_KEY`</code>	Google AI API key for Gemini 1.5 Flash

10.3 Local Development

```
# Build optimized assets
python3 scripts/build.py

# Start local server
python3 -m http.server 8080

# Visit http://localhost:8080
```

Note: The chatbot API (`/api/chat`) will return 404 on local dev server since Vercel serverless functions require the Vercel CLI or cloud deployment.

11. SEO & Performance

11.1 SEO Implementation

ELEMENT	IMPLEMENTATION
Title Tag	<code><title></code>
Meta Description	Comprehensive description with keywords
Meta Keywords	Targeted data engineering and AI terms
Meta Author	<code><author></code>
Semantic HTML	<code><h1></code> <code><h2></code> <code><h3></code> <code><h4></code> <code><h5></code> <code><h6></code> <code><p></code> <code></code> <code></code> <code><div></code>
Heading Hierarchy	Single <code><h1></code> <code><h2></code> , proper <code><h3></code> <code><h4></code> <code><h5></code> <code><h6></code> <code><h1></code> nesting
ARIA Labels	All interactive elements have <code><aria-label></code> attributes
Alt Text	All images have descriptive <code><alt></code> attributes
Favicon	Custom chatbot icon (<code>chatbot_icon.png</code>) on all pages

11.2 Performance Optimizations

OPTIMIZATION	IMPACT
Build pipeline minification	~30% file size reduction
Font preconnect	Faster Google Fonts loading
<code>`preload="metadata"`</code> on videos	No auto-download of video content
Canvas-based effects (vs DOM)	GPU-accelerated rendering
Sparkle throttling (40ms)	Prevents DOM flooding
<code>`requestAnimationFrame`</code>	Synced to display refresh rate
Lazy section reveals	Content rendered only when in viewport

12. Security Considerations

RISK	MITIGATION
API Key Exposure	Keys stored in Vercel environment variables, never in client code
XSS in Chat	Bot responses are inserted as <code>`textContent`</code> or sanitized HTML
CORS	API handler sets <code>`Access-Control-Allow-Origin: *`</code> for public access
Rate Limiting	OpenAI/Gemini enforce their own rate limits; no client-side throttle
localStorage Abuse	Data size capped (100 feedback entries, 50 FAQ items)

13. Rules & Coding Conventions

13.1 Development Rules

1. All edits go in `src/` — never edit root-level built files directly.
2. Run `python3 scripts/build.py` after every change to compile to production.
3. New binary assets must be added to `BINARY_FILES` in `build.py`.

4. **Sub-pages** (`projects.html` , `services.html` , `games.html`) **must include inline CSS overrides** to force `opacity: 1` on animated elements, since they don't load `script.js`.
5. **The chatbot system prompt** in `api/chat.js` must be updated whenever Arijeet's facts change.
6. **Keep local repository clean:** Ensure `.DS_Store` and other temp junk files are not tracked in Git by maintaining the `.gitignore` exclusions.
7. **Do not track nested duplicate folders:** Never check in duplicate subfolders (e.g. `Portfolio/`). Keep project files directly under the repository root.
8. **Git sync best practices:** When syncing local updates with a diverging remote history, check out a sync branch from `origin/main` , apply your changes, remove duplicate files, and commit. Fast-forward local `main` and push cleanly. Avoid force-pushing to prevent erasing commit history.

13.2 CSS Naming Conventions

- **BEM-inspired:** `.chatbot-toggle` , `.chatbot-header-info` , `.project-card`
- **State classes:** `.visible` , `.open` , `.recording` , `.riya-speaking` , `.riya-sleeping-mode`
- **Utility prefixes:** `.btn-icon` , `.meta-badge` , `.cert-icon`

13.3 JavaScript Conventions

- **ES6+ Classes** for all major components
 - **Event-driven communication** between modules (CustomEvent dispatch)
 - **Defensive coding:** Optional chaining (`?.`) and nullish checks throughout
 - **Performance:** `requestAnimationFrame` for animations, throttled event handlers
-

14. File Reference Index

FILE	TYPE	SIZE	PURPOSE
`src/index.html`	HTML	68 KB	Main portfolio page
`src/projects.html`	HTML	5.5 KB	Projects hub page
`src/services.html`	HTML	25 KB	Services marketplace
`src/games.html`	HTML	27 KB	Stress relief games
`src/style.css`	CSS	79 KB	Primary stylesheet
`src/swiss-override.css`	CSS	22 KB	Design overrides
`src/avatar3d.css`	CSS	3.3 KB	Avatar animations
`src/script.js`	JS	51 KB	Main application logic
`src/common.js`	JS	11 KB	Shared visual effects
`src/avatar3d.js`	JS	9.5 KB	Avatar SVG engine
`src/projects.js`	JS	14 KB	Projects page logic
`src/services.js`	JS	12 KB	Services page logic
`api/chat.js`	JS	7.3 KB	Serverless AI handler
`scripts/build.py`	Python	6.6 KB	Build pipeline
`vercel.json`	JSON	0.1 KB	Vercel deployment configuration
`.gitignore`	Text	0.1 KB	Git file exclusion rules
`package.json`	JSON	0.5 KB	Project metadata
`chatbot_icon.png`	PNG	98 KB	Favicon and chatbot icon
`videos/*.mp4`	MP4	~170 MB	Project demo videos

15. Appendices

Appendix A: Full Dependency List

This project has **zero npm runtime dependencies**. The only tooling dependency is **Python 3** for the build pipeline. All JavaScript is vanilla ES6+ with no frameworks, libraries, or bundlers.

Appendix B: Browser Compatibility

FEATURE	CHROME	FIREFOX	SAFARI	EDGE
CSS Custom Properties	✓	✓	✓	✓
IntersectionObserver	✓	✓	✓	✓
Web Speech API	✓	⚠ Partial	✓	✓
Web Audio API	✓	✓	✓	✓
Canvas API	✓	✓	✓	✓
CSS Grid/Flexbox	✓	✓	✓	✓

Appendix C: Color Palette

NAME	HEX	USAGE
Primary Cyan	`#00f0ff`	Accents, links, glowing effects
Accent Purple	`#8b5cf6`	Secondary accents, gradients
Background Navy	`#07131d`	Page background
Card Background	`#0d1b2a`	Card surfaces
Text Primary	`#e2e8f0`	Body text
Text Secondary	`#94a3b8`	Muted labels
Success Green	`#06d6a0`	Sparkle color
Alert Pink	`#f72585`	Sparkle color
Gold	`#ffd166`	Sparkle color

Appendix D: API Response Format

Success:

```
{  
  "reply": "Arijeet is a data engineer with 4+ years at Bosch and Microsoft..."  
}
```

Error:

```
{  
  "error": "Server AI endpoints currently offline"  
}
```

This document was generated as part of the Portfolio Web Application project. For the latest version of the source code, refer to the `src/` directory.